Module: CSS and Responsive Design

Shared topic (Python FS + Java FS).

Previous: HTML
Next: JavaScript DOM Forms

> **Module Outcome:**
> - Style a card and a form without inline `style=` soup
> - Make a layout that still works on a phone (flex / media queries)
> - Separate look from HTML so Swiggy and Employee Management Portal can share a palette
>
> **CSS** controls **how HTML looks**: colors, fonts, spacing, layout. HTML says "this is a button"; CSS says "blue, rounded, centered."

Method	When
Inline `style=""`	Avoid — hard to maintain
Internal `<style>`	Single-page demos only
External `.css`	**Real projects**

Cascade: More specific rules win; later rules can override earlier ones.

Full-stack: `HTML + CSS + JS → Django/React`. Django serves CSS from `static/css/`. React uses component CSS or Tailwind — **Flexbox and selectors transfer directly**.

> **Where used?**
> - CampusCart / **Swiggy** — product card grid, promo banner, responsive nav
> - **Employee Management Portal** — employee directory cards, payslip layout
> - **Apollo Hospitals** — appointment board, doctor cards
> - **HDFC Bank** — account summary tiles, transfer confirmation
>
> CampusCart remains the landing-page lab skin. Treat it as the college **Swiggy** storefront.

Important Concepts

1. Linking CSS

```
<link rel="stylesheet" href="css/styles.css">
```

```
* { box-sizing: border-box; }

body {
  font-family: system-ui, Roboto, sans-serif;
  line-height: 1.6;
  color: #1a1a2e;
  margin: 0;
```

```
  background: #f8f9fa;
}
```

Try these (2 programs):

Program 1 — Linked stylesheet

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Link CSS</title>
  <link rel="stylesheet" href="css/styles.css">
</head>
<body>
  <h1>CampusCart</h1>
  <p>Styles come from an external file.</p>
</body>
</html>
```

```
/* css/styles.css */
* { box-sizing: border-box; }
body {
  font-family: Georgia, "Times New Roman", serif;
  margin: 0;
  color: #1a1a2e;
  background: #f0f4f8;
}
```

Program 2 — Reset margins

```
body { margin: 0; line-height: 1.6; }
h1, p { margin: 0 0 1rem; }
```

2. Selectors

```
h1 { font-size: 2rem; }

.btn-primary {
  background: #2563eb;
  color: white;
  padding: 0.75rem 1.5rem;
  border: none;
  border-radius: 8px;
  cursor: pointer;
}

#hero-title { letter-spacing: -0.02em; }

nav a { text-decoration: none; color: #1e40af; }
nav a:hover { text-decoration: underline; }

input:focus { outline: 2px solid #2563eb; }
```

Specificity (lite): inline > `#id` > `.class` > element.

Note:
- Overusing `!important` — fix specificity instead (class vs id vs element).
- No focus/hover states — add `:hover` and `:focus-visible` for links, buttons, and inputs.

Try these (2 programs):

Program 1 — Class button

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Selectors</title>
  <style>
    .btn-primary {
      background: #0f766e;
      color: white;
      padding: 0.75rem 1.5rem;
      border: none;
      border-radius: 8px;
      cursor: pointer;
    }
    .btn-primary:hover { background: #115e59; }
  </style>
</head>
<body>
  <button class="btn-primary">Order Thali — ₹90</button>
</body>
</html>
```

Program 2 — Nav link hover

```
nav a { text-decoration: none; color: #0f766e; }
nav a:hover { text-decoration: underline; }
input:focus { outline: 2px solid #0f766e; }
```

3. Box model

```
margin → border → padding → content
```

```
.card {
  width: 280px;
  padding: 1.25rem;
  margin: 1rem;
  border: 1px solid #e5e7eb;
  border-radius: 12px;
  background: white;
}
```

DevTools → Computed → box diagram.

Note:
- No `box-sizing: border-box` — add a global border-box rule so padding/border stay inside the set width.

Try these (2 programs):

Program 1 — Swiggy product card box

```
.card {
  width: 280px;
  padding: 1.25rem;
  margin: 1rem;
  border: 1px solid #d1d5db;
  border-radius: 12px;
  background: white;
}
```

```
<article class="card"><h3>Filter Coffee</h3><p>₹25</p></article>
```

Program 2 — box-sizing demo

```
* { box-sizing: border-box; }
.box {
  width: 200px;
  padding: 20px;
  border: 5px solid #0f766e;
  /* total width stays 200px with border-box */
}
```

4. Typography and CSS variables

```
:root {
  --primary: #2563eb;
  --text-muted: #6b7280;
}

h1 { font-size: clamp(1.75rem, 4vw, 2.75rem); }
p.lead { color: var(--text-muted); }
section { padding: 3rem 1rem; max-width: 1100px; margin: 0 auto; }
```

Use **rem** for scalable spacing.

Try these (2 programs):

Program 1 — Brand variables

```
:root {
  --brand: #0f766e;
  --muted: #6b7280;
}
h1 { color: var(--brand); font-size: clamp(1.75rem, 4vw, 2.5rem); }
.lead { color: var(--muted); }
```

Program 2 — Readable section

```
section {
  padding: 3rem 1rem;
  max-width: 1100px;
  margin: 0 auto;
}
p { font-size: 1rem; }
```

5. Flexbox

```
.site-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 1rem 2rem;
  background: white;
  box-shadow: 0 1px 3px rgba(0,0,0,0.08);
}

.nav-links {
  display: flex;
  gap: 1.5rem;
  list-style: none;
  margin: 0;
  padding: 0;
}

.product-grid {
  display: flex;
  flex-wrap: wrap;
  gap: 1.5rem;
  justify-content: center;
}

.product-card { flex: 1 1 260px; max-width: 320px; }
```

Property	Purpose
`display: flex`	Enable flex container
`justify-content`	Main axis alignment
`align-items`	Cross axis alignment
`gap`	Space between children
`flex-wrap`	Wrap to next line

Note:
- Flex without wrap — add `flex-wrap: wrap` so cards/nav items can drop to the next line.
- Tables for layout — use Flexbox for rows and grids of cards.

Try these (2 programs):

Program 1 — Header row

```
.site-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 1rem 2rem;
  background: white;
}
.nav-links {
  display: flex;
  gap: 1.5rem;
  list-style: none;
```

```
  margin: 0;
  padding: 0;
}
```

Program 2 — Swiggy product grid

```
.product-grid {
  display: flex;
  flex-wrap: wrap;
  gap: 1.5rem;
  justify-content: center;
}
.product-card { flex: 1 1 260px; max-width: 320px; }
```

6. Media queries — mobile-first

```
.nav-links { flex-direction: column; gap: 0.75rem; }

@media (min-width: 768px) {
  .nav-links { flex-direction: row; }
  .hero { display: flex; align-items: center; gap: 2rem; }
}

@media (min-width: 1024px) {
  section { padding: 4rem 2rem; }
}

img { max-width: 100%; height: auto; display: block; }
```

Note:
- Fixed widths on mobile — prefer `%`, flex, and `max-width` so layouts shrink.
- Missing viewport meta — required from the HTML module for mobile-first CSS to behave.

Try these (2 programs):

Program 1 — Stack then row

```
.nav-links { flex-direction: column; gap: 0.75rem; }

@media (min-width: 768px) {
  .nav-links { flex-direction: row; }
}
```

Program 2 — Fluid images

```
img { max-width: 100%; height: auto; display: block; }

@media (min-width: 1024px) {
  section { padding: 4rem 2rem; }
}
```

7. Landing page structure

```
<header class="site-header">
  <div class="logo">CampusCart</div>
  <ul class="nav-links">
```

```
    <li><a href="#menu">Menu</a></li>
    <li><a href="#pricing">Pricing</a></li>
  </ul>
</header>

<section class="hero">
  <div class="hero-content">
    <h1 id="hero-title">Campus food, delivered fast</h1>
    <p class="lead">Order from your canteen — skip the queue.</p>
    <a class="btn-primary" href="#menu">Browse Menu</a>
  </div>
</section>

<section id="menu" class="product-grid">
  <article class="product-card card">
    <h3>Veg Thali</h3>
    <p class="price">₹89</p>
    <button class="btn-primary">Add to cart</button>
  </article>
</section>
```

Style with Flexbox header, flex-wrap card grid, responsive nav column→row at 768px.

Try these (2 programs):

Program 1 — Mini hero HTML+CSS

```
<header class="site-header">
  <div class="logo">CampusCart</div>
  <ul class="nav-links"><li><a href="#menu">Menu</a></li></ul>
</header>
<section class="hero">
  <h1>Campus food, delivered fast</h1>
  <p class="lead">Skip the canteen queue in Hyderabad.</p>
  <a class="btn-primary" href="#menu">Browse Menu</a>
</section>
```

Program 2 — Swiggy three cards

```
<section id="menu" class="product-grid">
  <article class="product-card card"><h3>Veg Thali</h3><p>₹90</p></article>
  <article class="product-card card"><h3>Masala Dosa</h3><p>₹60</p></article>
  <article class="product-card card"><h3>Filter Coffee</h3><p>₹25</p></article>
</section>
```

Step-by-Step Lab

1. Create `css/styles.css`; link from HTML; remove inline styles
2. Add global `box-sizing: border-box` and body defaults
3. Flexbox header/nav with hover states
4. Three product/project cards with gap
5. Media query at 768px: horizontal nav, optional hero two-column
6. Test at 375px in DevTools — fix horizontal scroll
7. Lighthouse contrast check on buttons

Practice Exercises

Basic

1. Style `h2` with color + bottom border; create `.btn-secondary` outline style.
   Model answer:

```
h2 {
  color: #1a3a4a;
  border-bottom: 2px solid #1a3a4a;
  padding-bottom: 0.25rem;
}
.btn-secondary {
  background: transparent;
  border: 2px solid #1a3a4a;
  color: #1a3a4a;
  padding: 0.5rem 1rem;
}
```

2. `.card` with rem padding/margin; center div `max-width: 600px; margin: 0 auto`.
   Model answer:

```
.card {
  padding: 1.25rem;
  margin: 1rem 0;
}
.centered {
  max-width: 600px;
  margin: 0 auto;
}
```

3. Nav link `:hover` effect.
   Model answer:

```
nav a:hover {
  color: #0b6e4f;
  text-decoration: underline;
}
```

Intermediate

1. Flex footer: copyright left, links right; stack on mobile.
   Model answer:

```
.footer {
  display: flex;
  justify-content: space-between;
  gap: 1rem;
}
@media (max-width: 600px) {
  .footer { flex-direction: column; align-items: flex-start; }
}
```

2. Three-column cards → one column below 768px.
   Model answer:

```
.grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 1rem;
}
@media (max-width: 768px) {
  .grid { grid-template-columns: 1fr; }
}
```

3. Hero with `clamp()` heading; CSS variables for primary color.
   Model answer:

```
:root { --primary: #0b6e4f; }
.hero h1 {
  color: var(--primary);
  font-size: clamp(1.75rem, 4vw, 3rem);
}
```

4. Style HTML table (zebra rows, header background).
   Model answer:

```
thead { background: #1a3a4a; color: #fff; }
tbody tr:nth-child(even) { background: #f0f4f5; }
```

Interview-style

1. Box model and `box-sizing: border-box`?
   Model answer:
   Content + padding + border (+ margin outside) make up layout size. `border-box` includes padding and border inside the declared `width`, so sizing is more predictable.
2. Flexbox vs Grid — when Flexbox? (1D: nav, toolbars)
   Model answer:
   Use Flexbox for one-dimensional alignment (nav rows, toolbars, centering). Use Grid when you need two-dimensional rows and columns together.
3. Mobile-first CSS?
   Model answer:
   Write base styles for small screens, then add `min-width` media queries for larger layouts. It matches mobile usage and avoids fighting desktop-first overrides.
4. Specificity: `#nav .link` vs `.nav a`?
   Model answer:
   `#nav .link` wins: ID (100) + class (10) beats two type/class-level selectors like `.nav a` (class + element). Higher specificity overrides lower when both match.
5. Django static CSS path?
   Model answer:
   Put CSS under an app or project `static/` folder and load with `{% load static %}` then `<link href="{% static 'css/site.css' %}" …>`. Run `collectstatic` in production.

Mini-Project (Assignment): Responsive Landing Page

Lab kit (build here): labs/22_css_responsive_cards/

Option A — Portfolio: Flex header, hero, 3+ project cards, footer — works at 375px and 1280px
Option B — CampusCart: Product grid (4+ items), features section, promo banner

What to build

`index.html`, `css/styles.css`, mobile + desktop screenshot.

Stretch

- Add CSS variables for brand color and muted text.
- Add a second breakpoint at `1024px`.
- Prefer `rem` spacing over raw `px` for padding.

Done when

- [ ] Lab kit responsive cards ticket completed
- [ ] External stylesheet linked (not only inline styles)
- [ ] Flex layout for header and card grid
- [ ] Page usable at ~375px width without horizontal scroll

Interview Q&A

Shared with students after concepts and the mini-project. Use for review and interview prep — questions and model answers are included.

1. Why separate CSS from HTML?
2. Mobile-first vs desktop-first for Indian users?
3. When is inline CSS acceptable? (email templates — rare)
4. How will React handle these styles?
5. CSS vs Bootstrap — when build custom?

Q1. CSS and HTML relationship?
A: HTML structure; CSS visual presentation — layout, colors, typography.

Q2. Box model?
A: Content surrounded by padding, border, margin; box-sizing controls width calculation.

Q3. Flexbox use?
A: One-dimensional layouts — nav bars, card rows, centering.

Q4. Mobile-first?
A: Default small-screen CSS; min-width media queries enhance for larger viewports.

Q5. CSS specificity example?
A: `#header .nav a` beats `.nav a`; avoid `!important` unless necessary.